

摘要：

DeepSeek发布首款Agent产品Harness，并非Codex类产品，而是一套"可重组、可回放的Agent运行时"。其核心是"一切皆插件"，通过"Agent预设"决定工具与能力，将模型、界面等均可替换组合。它把Agent视为基础设施，让用户自行组装专属Agent，而非仅使用通用产品。

就在刚刚，DeepSeek 正式发布了首款 Agent 产品，DeepSeek Harness。

早在 V4 Flash 正式版更新日志里，除了直接对标 Claude Opus 模型的 benchmark 成绩，还有一行说明写着「**正式版 DeepSeek-V4-Flash 使用 DeepSeek Harness 极简模式（即将发布）作为框架进行测试。**」

APP SO 在发布之前拿到了 DeepSeek Harness 的内测资格，管理项目、长任务协作、多 Agent 编排、上下文管理以及联网搜索和 Skill 等等，这些本地 Agent 工作台该有的功能，DeepSeek Harness 全部都有。



DeepSeek Harness 自己描述的与 Codex 的区别

但如果只是用「DeepSeek 版 Codex」来形容它又肯定不够，**自定义的 Agent 预设，可重建的会话，以及强大的插件功能**，都让 DeepSeek Harness 变得和传统的 Codex 类产品不太一样。

当 DeepSeek 也开始争夺模型输出之后的 Agent 执行层，这个姗姗来迟的 Harness 除了可以让手上本就「物美价廉」的 DeepSeek 模型更好用，更重要的是，它或许会改变我们使用 Agent 的习惯。

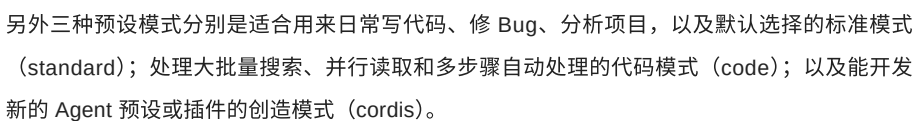
先来认识一下 DeepSeek Harness

以下功能和界面等内容均来自内测版，可能与正式版略有出入，以正式发布的版本为主。

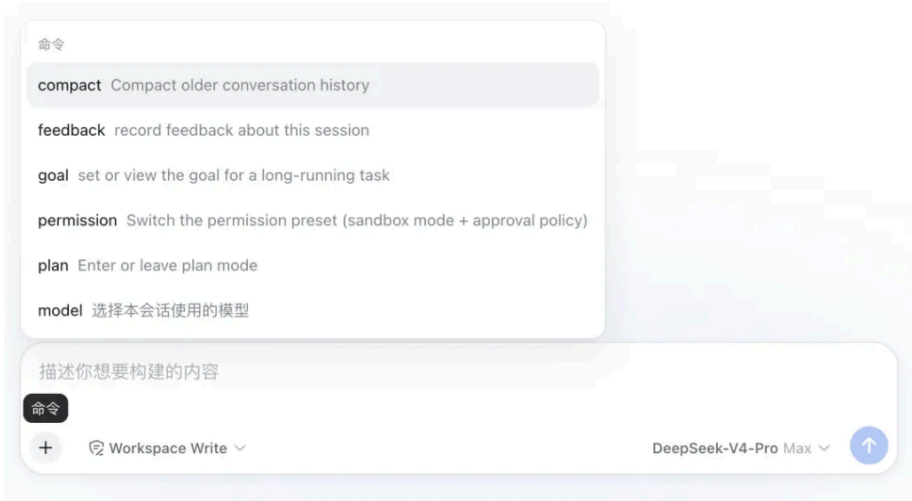
第一眼看和市面上大多数的本地 Agent 产品没什么两样，左侧边栏从上到下依次是「新建会话」、「工作区」，以及不同工作区/文件夹内的多个会话。



在 V4 Flash 正式版中提到的极简模式（minimal），正是 Harness 内置的四种 Agent 预设之一，它适合用于简单修改、测试最小 Agent，没有压缩、搜索、Skill、计划和子 Agent 这些功能。



另外三种预设模式分别是适合用来日常写代码、修 Bug、分析项目，以及默认选择的标准模式（standard）；处理大批量搜索、并行读取和多步骤自动处理的代码模式（code）；以及能开发新的 Agent 预设或插件的创造模式（cordis）。



标准模式、代码模式和创造模式，都包含有对话压缩、目标、计划等命令，而极简模式下，只有目标命令。

用创作模式自定义 Agent 预设

对于什么是 Agent 预设，以及几款 Agent 预设的具体区别，我们还能在设置页面看到详细的情况。

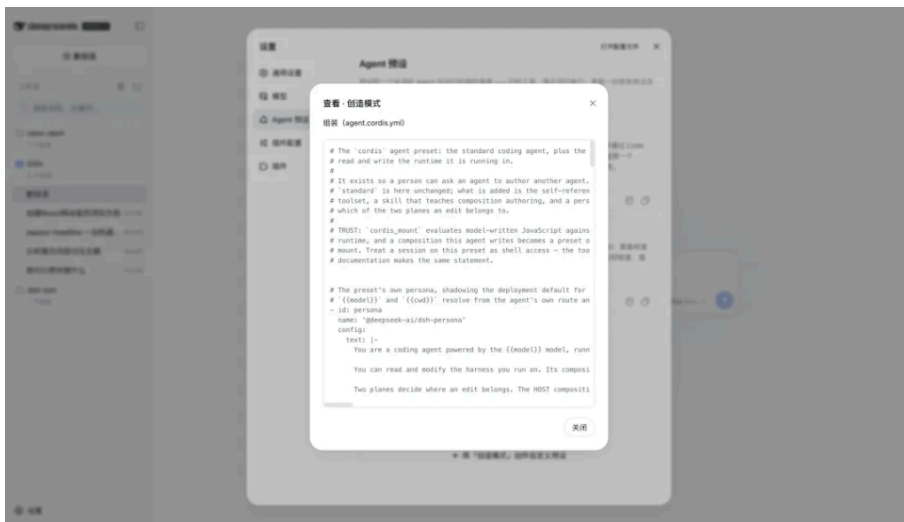


预设即一个会话的 Agent 所运行的插件组装 —— 它的工具、提示词与能力。复制一份既有预设改成自己的，或用「创造模式」让 Agent 帮你创建。

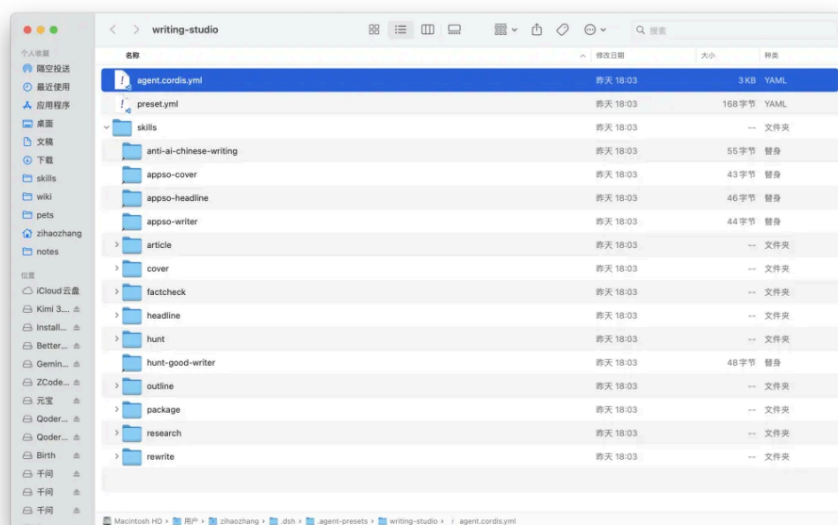
如果说模型等于大脑，决定基础的推理和理解能力；Skill 等于一份操作手册，告诉模型遇到某类任务时，具体怎么做；Tool / 插件 相当于软件和权限，它决定了模型能否搜索、改文件、运行命令、收发邮件等。

那么 Agent 预设就是一个岗位+工作环境，身份、长期规则、可用工具、工作方式，都由 Agent 预设决定。

一个 Agent 预设通用由一份配置文档构成，文档内把 Agent 的系统提示词、工具、上下文压缩和多 Agent 能力完整组合起来。

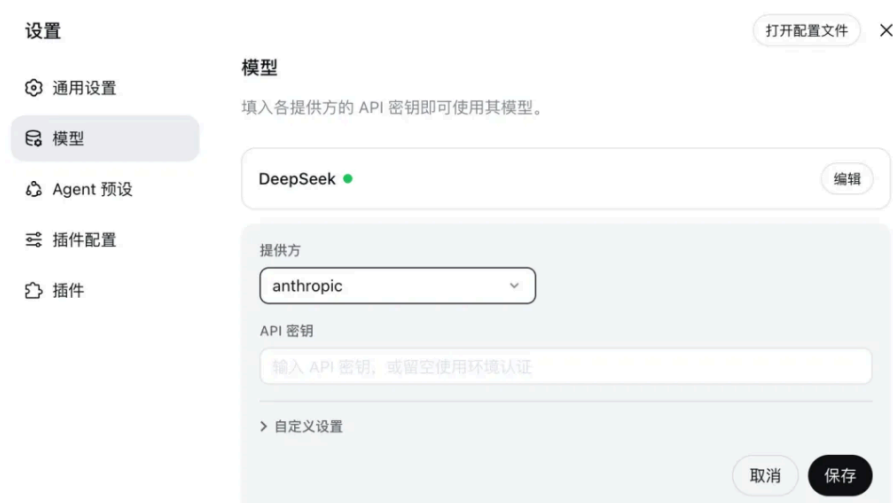


我们也新建了一个专门用于写作的 Agent 预设，在这份预设内，不仅可以使用文件进行配置，还能引入文件夹，加入不同的 Skill，来让自己的 Agent 更加完善。



DeepSeek Harness 内也提供了「用创作模式自定义预设」的方法，就像大多数 Agent 用对话创建 Skills 一样，我们可以一步步要求 Harness 创建一个自己的 Agent 预设。

在设置页面，我们还能看到模型的配置，DeepSeek Harness 允许我们接入不同的第三方大模型，默认提供方包括 Kimi、OpenAI、Anthropic、Google 等将近 40 个大模型厂商。

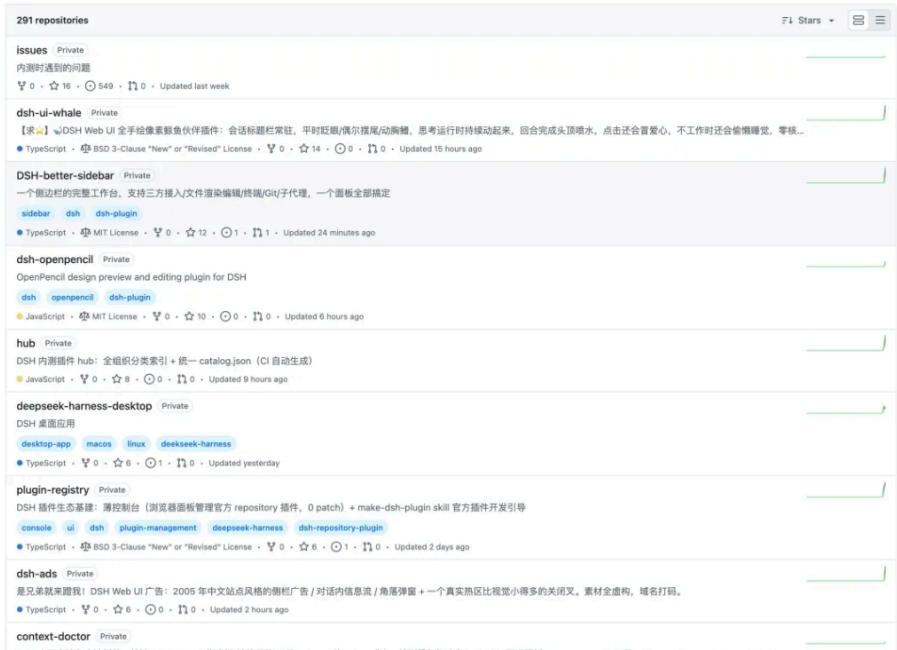


一切都是插件化运行

最后一项设置是 DeepSeek Harness 这次的重头戏——插件，DeepSeek Harness 在内测产品中介绍里写着「Everything is a plugin」，一切皆插件。



这个插件有多离谱，在内测的项目仓库里，参与内测的用户短短几天的时间就已经开发了约 300 个插件。

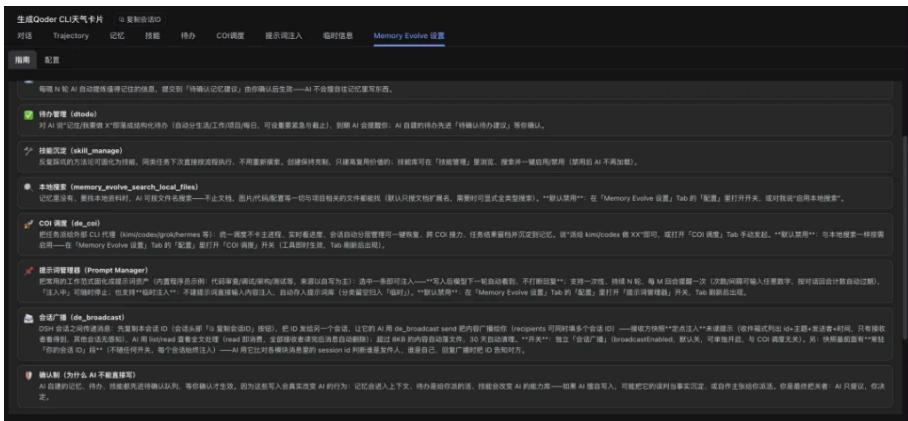


有的插件能把 DeepSeek Harness 整个的工作界面修改，有的可以像 Codex 一样接入自定义桌面宠物，还有针对对话界面的优化，以及为 DeepSeek Harness 带来「跨会话长期记忆 + 后台自我进化」能力的纯插件实现。

虽然最底层的记忆依然是本地文件，但这套插件并没有把「长期记忆」做成传统的向量数据库 + RAG，而是通过本地文件持久化 + 分层上下文注入 + LLM 自我整理，形成的一套完整系统。

```
~/dsh/memories/
├── MEMORY.md
├── USER.md
├── SUGGESTIONS.jsonl
├── MEMORY-archive.md
├── USER-archive.md
├── scratch.md
├── prompts.json
├── prompt-injections.json
├── skills-state.json
├── TODOS-life.md
├── TODOS-work.md
├── TODO-archive.md
├── models.json
├── session-bookmarks.json
├── daily/YYYY-MM-DD.md
├── daily/YYYY-MM-DD.todo.md
├── projects<cwd-hash>/
│   ├── MEMORY.md
│   ├── KEY.md
│   ├── KEY-archive.md
│   └── TODOS.md
└── # 全局事实 ($ 分隔条目格式)
    # 用户档案
    # 待确认建议队列
    # 归档记忆 (不注入, 可转正)
    # 归档用户 (不注入, 可转正)
    # 临时信息便签 (自由文本, 非 $ 格式, 随意编辑)
    # 提示词库 (提示词管理器: CRUD + 分类 + 统计)
    # 注入轨 (活跃注入条目 + 剩余回合计数)
    # 技能禁用列表 + 自定义技能目录 (技能管理器)
    # 生活待办 ($ 分隔 + 文件夹 tag 说明)
    # 工作待办 (跨项目)
    # 待办建议归档 (可转正回对应待办轨)
    # 模型设置 (模型设置模块: 每模型启用/备注/思考等级: 仅插件侧, 不影响 DSH)
    # 会话书签 (书签模块: 按会话隔离、按轮 seq 定位, 含分支数据)
    # 每日日志 (按天分文件)
    # 每日待办 (与日志分离, 按天分文件)
    # 项目记忆 (每个工作目录独立)
    # 项目日志 (每回合追加)
    # 项目关键记忆 (长期事实, 注入上下文)
    # 项目关键记忆归档 (不注入, 可转正)
    # 项目待办 (cwd 隔离)
```

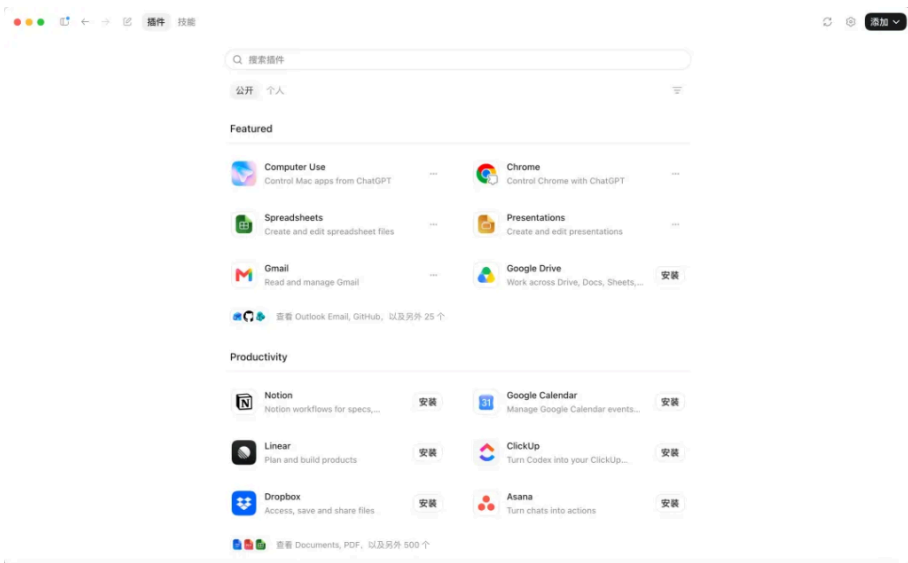
插件的能力还在于能实现模型的自我进化，定期让 LLM 回头审视自己的完整工作上上下文，把临时经验压缩成长久知识。



但无论是记忆插件，还是皮肤插件，这些都只是插件能力的冰山一角。

在 DeepSeek Harness 的开发文档里，从架构层面来说，插件几乎就是 DeepSeek Harness 的主打，模型、工具、策略、存储、上下文和界面都能通过插件进行替换或组合。

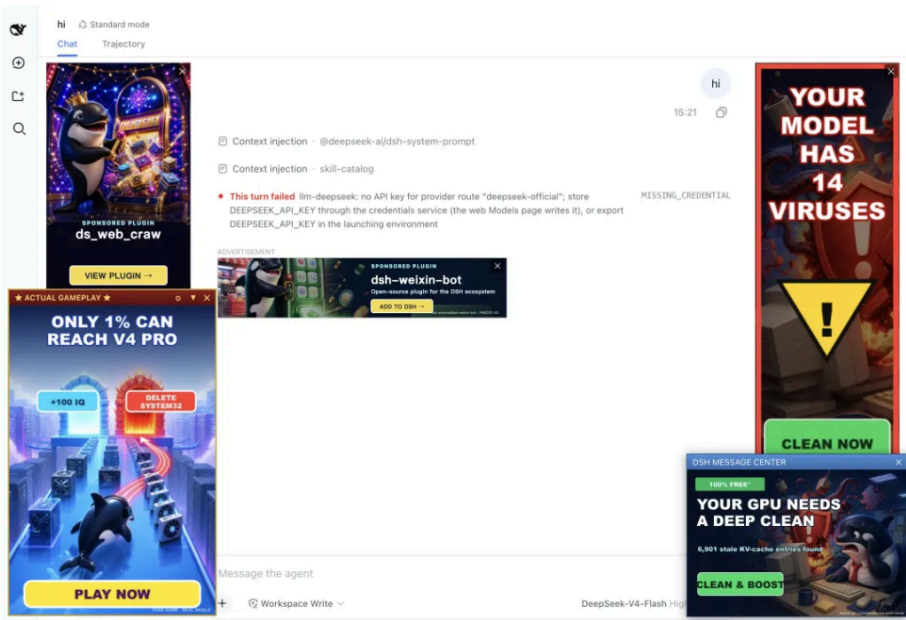
通常我们理解的插件是给当前的产品增加一些小功能，就像 VS Code 里有着丰富的插件系统，或者是 Chrome 浏览器，我们能安装不同的插件，增加浏览器对应的能力。



Codex 应用内也有大量的插件，像是计算机使用、Chrome、Notion 等等对应不同的工具和服务。

但 DeepSeek Harness 这次给出的插件，是在 Agent 的大脑、工具箱、规则和界面里加能力，它更深入、更自由。

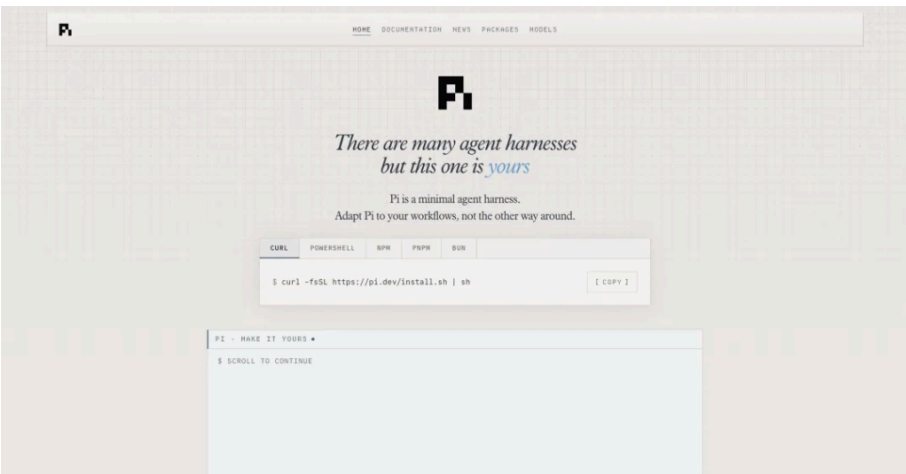
如果你嫌 DeepSeek 官方界面太简洁，不妨加点广告。



举个例子，一个 Agent 的「标题生成插件」，通常是在对话里增加一个按钮或命令，点击后调用 AI 根据已有命令处理。

一个 DSH 的「写作插件」，则可能同时做到：给 Agent 增加 /headline 等能力；注册一个模型可以自主调用的标题工具；换掉系统提示词和写作规则；接入我们的各类 Skills；增加网页检索或外部 MCP；保存文章素材和长期偏好；在 DeepSeek 的交互页面中增加写作面板；限制这个 Agent 可以使用哪些 Shell、文件或网络工具……等等。

市面上插件功能同样强大的产品可能是 Pi Agent，它也可以通过 Extension 改写工具、模型请求、会话行为和终端界面。



Pi 让一个 Coding Agent 可以任意扩展；DSH 则试图让整个 Agent 产品都由插件重新组合。



Codex、Pi 与 DSH

插件系统对比

同样支持扩展，控制的层级不同

01 Codex 产品扩展

插件内容	Skill · MCP · App
控制层级	外部能力与服务
管理方式	平台目录 · 统一安装

把能力接进产品

02 Pi 代码扩展

插件内容	Extension · Skill · Prompt · Theme
控制层级	工具、事件与会话行为
作用范围	全局 · 项目

把行为交给扩展

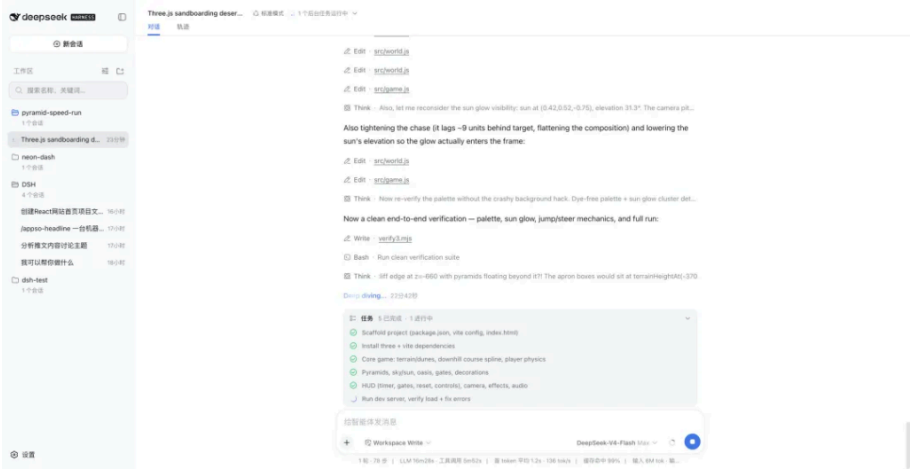
03 DSH 运行时组合

插件内容	模型 · 工具 · 策略 · 存储 · UI
控制层级	Agent 的运行方式
作用范围	Profile · Preset · Agent

用插件组合运行时

用起来怎么样

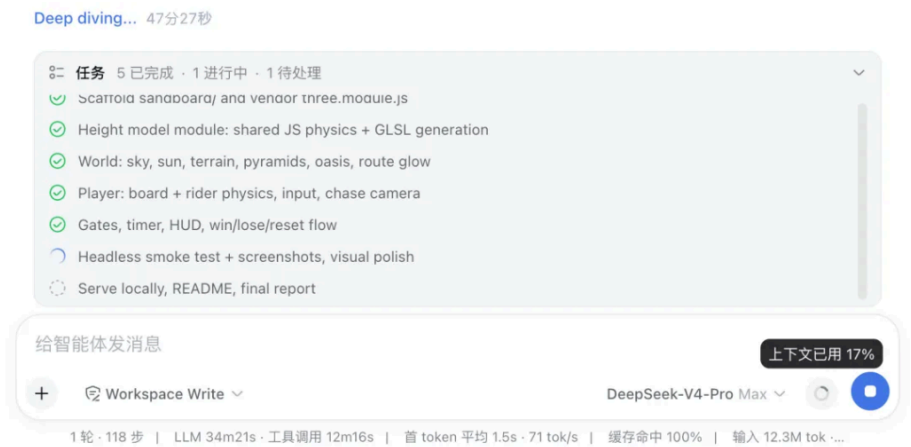
很难想象吧，以快著称的 DeepSeek，有一天运行一项编程任务的时间也会来到半个多小时。



为了观察执行层能给同一个模型带来多大变化，我们让 DeepSeek-V4-Flash 分别在 DeepSeek Harness、Reasonix 和 Codex 中完成同一个 Three.js 滑沙游戏。

先看使用 V4 Flash 在 DeepSeek Harness 中的表现，基本上没有太多可以挑剔的 Bug，玩家一开始就在滑行，完全遵守核心玩法，穿过下坡途中的每个门，最终抵达沙漠绿洲。

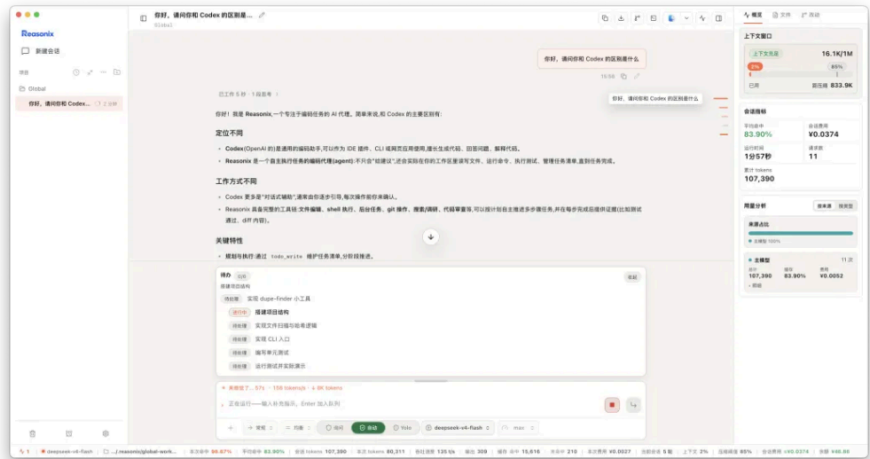
而使用刚刚上新的 V4 Pro，用时更长，表现反而更没那么好。



虽然界面似乎有更加精美了一点，太阳光沙漠都更自然，但是金字塔等元素又很抽象。

使用 DeepSeek V4 Pro 正式版，在 DeepSeek Harness 内生成

Reasonix，据说是最适合 DeepSeek 的第三方 Agent，我们让它处理了同一个任务，看看 V4 Flash 正式版模型，在不同的 Harness 应用中，表现会有什么不同。

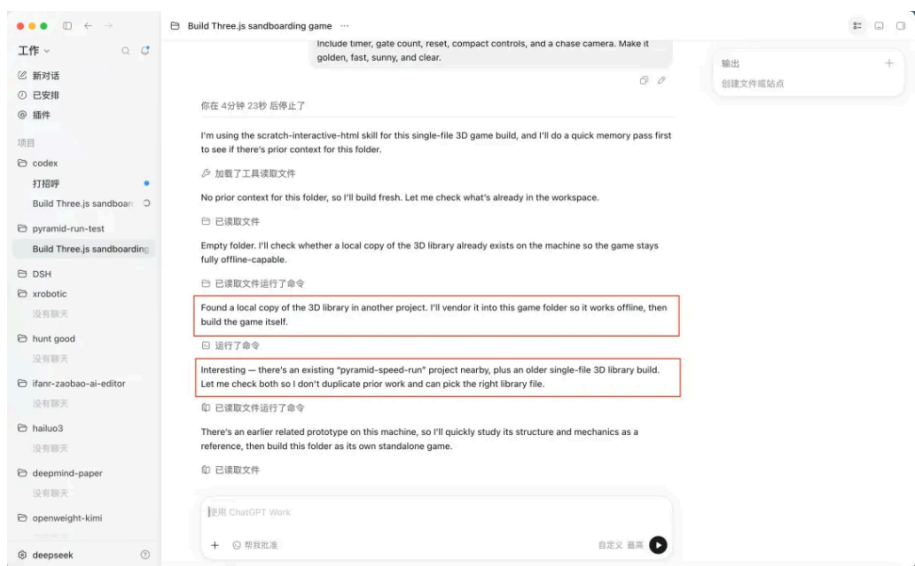


同样是最高等级的推理，差别真的很大。

首先是整体画面的美感就完全不如 DeepSeek Harness 所交付的「金黄、快速、阳光明媚、清晰明朗」，其次虽然游戏同样能玩，但是太过于简陋，渲染的金字塔、人物、天空都很粗糙，不像是一款 3D 游戏。

而如果是 Codex+DeepSeek V4 Flash 的组合呢，一开始我们在本地新建了一个文件夹，Codex 的处理方式很聪明，他说他自己先扫描了一遍本地文件夹，发现了另一个项目中存在一个 3D 库的复制文件。

接着他又找到了我使用 DeepSeek Harness 创建的 pyramid-speed-run 项目，他说他要先看看那个是怎么处理 3D 库文件的，但是他不会直接复制，然后把那个项目作为一个原型参考，开始构建自己的项目。



一个是允许他继续参考做出来的网页，一个是我们新开了一个对话，让 Codex 完全依靠自己的工具处理的网页。

两个版本似乎都比用 Reasonix 要更好，和 DeepSeek Harness 版本相比，似乎还是 Harness 版本给人沉浸感更强。

「受污染」版把色彩调好的同时，但是金字塔完全不像是金字塔。而从 0 开始做的版本确实有 Codex 的感觉，整体画面更明亮，也更简单。

就像 V4 Flash 使用极简模式的 DeepSeek Harness 进行评测能得到更好结果。

这个单次案例虽然无法证明 Harness 在所有任务上更强，但至少说明：**当模型完全相同时，Harness 提供的工具、提示词、上下文组织和执行策略，已经足以显著改变最终产物。**

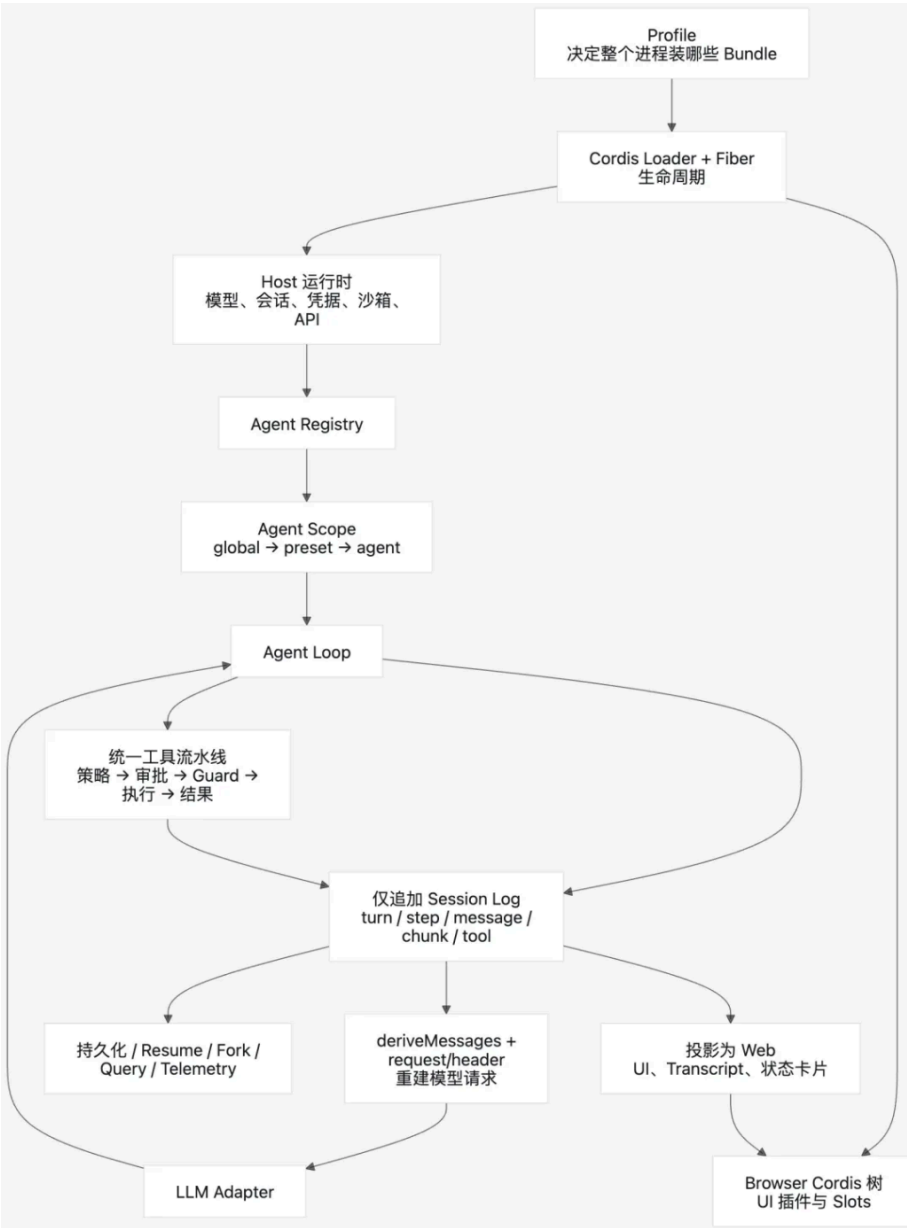
它不是 DeepSeek 版 Codex

查看 DeepSeek Harness 仓库中大量的 Markdown 说明文档，以及使用指南等内容，可以很确定地说，**DSH 不是「DeepSeek 版 Codex」**，它更接近一套可重组、可回放的 Agent 运行时。

DeepSeek Harness 真正特别的地方，是把「**插件生命周期、每个 Agent 的能力组合、工具执行、会话日志、Web UI**」接进了同一套内核。



从底层的技术配置到如何处理用户交互、AI 交互，DeepSeek Harness 都有自己独特的一套流程。



DeepSeek Harness 最重要的插件功能，根据其文档介绍，它并不只是一个 API 接口，每个工具、模型适配器、策略、提示词、存储、UI 区域都可以是插件。

Cordis 入门

[English](#) | [中文](#)

Cordis 是 DeepSeek Harness SDK 底层以 vendor 方式引入的插件框架。本文介绍 harness 插件作者在阅读[子系统页面](#)上生成的服务/事件参考之前需要了解的 Cordis 核心概念；[Cordis 教程](#)则通过实践逐一讲解这些概念。vendor 源码与同步流程见[vendor/README.md](#)。

五个核心概念

- 插件是实现 Service 的对象。它可以是一个带有可选 inject 和 apply(ctx) 字段的函数，也可以是一个 Service 子类，其生命周期由 Cordis 挂载到当前上下文中。
- 上下文是服务的容器。一个服务占据一个稳定的 ctx.<key>（如 ctx.tools、ctx.llm、ctx.sessions）；其他插件通过 key 查找服务，而非导入具体实现。
- 通过 inject 声明服务依赖。插件声明所需的服务后，会等待这些服务就绪才启动；加载顺序通过服务依赖表达，而非手动编排启动序列。
- 类型化事件用于通信。服务通过 TypeScript 声明并注册事件名，然后以 emit、waterfall（瀑布式事件）、parallel 或 serial 方式分发，分别对应监听者观察、包装、并行扇出或按序执行。
- 注册是可逆的副作用。提示词片段、工具 schema、适配器、提供方和监听器通过 ctx.effect() 或 ctx.on() 安装，reload 和 teardown 时会按预期撤销。

就像我们看到，已经有内测用户能通过插件，修改 DeepSeek Harness 的主界面。

其次是 Profile 和 Preset 构成的两级组合系统，Profile 决定整个 DSH 进程怎么运行，例如 Web、Headless、安装哪些 Bundle，它本质上是有序 cordis.patch.yml 配置层。



Agent Preset（Agent 预设）决定某个会话里的 Agent 能看到哪些工具、提示词、Skill、子 Agent 和工作流。Preset 的作用域按 agent → preset → global 解析。

preset/: 按会话组装 agent

English | 中文

agent preset 是一个目录，其中放置一份 agent.cordis.yml。把它挂载到某个 agent（智能体）的 scope 上下文之下，该会话就获得自己的工具与提示词段落，而其他在运行的会话各自保持不变，因此一个进程可以同时运行多个组装方式不同的 agent。

包	职责	ctx 键
agent-presets/	preset 词汇、在受信任目录与用户自建目录上的文件系统发现，以及带校验的按 agent 挂载	ctx.agentPresets
persona/	把 agent 人设做成可组装的行，使 preset 不止能改工具、也能改身份	—

部署交付哪些 preset，看 [apps/cli/config/agent-presets/](#)——一个 preset 一个目录，那份目录列表就是清单。在这里再列一遍只会多出一份需要同步的名单，而且总是它先过时。

本组假定的组装划分是：注册表与跨会话设施是进程单例，留在宿主组装中；preset 只承载单个 agent 对它们的贡献。若 preset 中某一行发布了进程级全局服务，挂载时即被拒绝，而不是留到与下一个会话相撞。

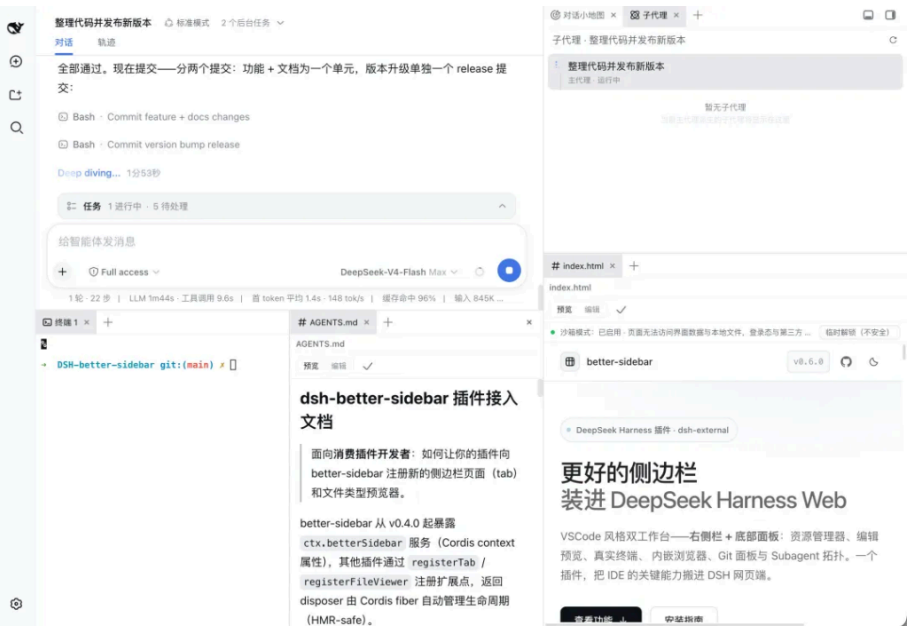
设计详见 [按会话组装 agent preset 的 Agent Note](#)。

根据 Preset 作用域机制，意味着同一个进程中，可以同时运行写作 Agent、编码 Agent、研究 Agent，而且各自看到不同的工具和指令，不必启动四套服务。

另一个有价值的技术点是「模型可见 ⇌ 已记录」。

普通聊天工具往往只保存最终消息，DSH 保存的是完整事件流：turn/start、step/start 用户消息、实际请求使用的模型、系统提示词、工具定义、原始流式等等内容。

同时甚至上下文压缩也不会删除原始历史，只是用 replacement 事件改变模型此后看到的「表面」。



DSH-better-sidebar 插件，将 DeepSeek Harness 界面修改为多个侧边栏的 UI

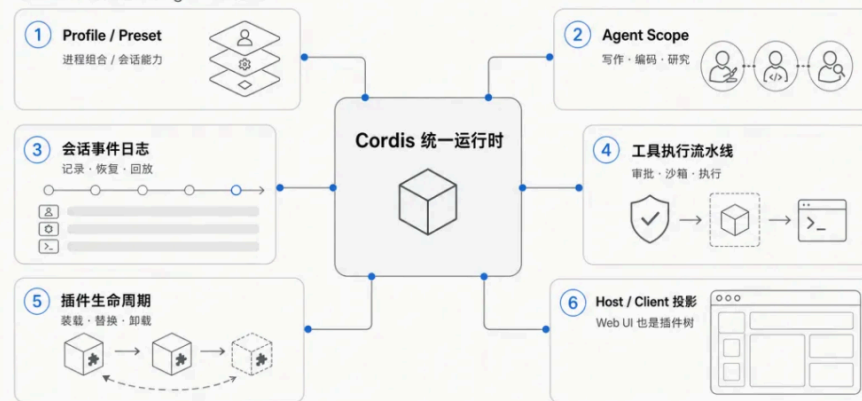
还有采用「先记录意图，再执行副作用」的 Agent Loop，以及所有工具共用一条执行内核。即模型可以写一段 TypeScript，把多个工具调用编排成程序，但中间数据留在运行环境里，只有最终结果进入模型上下文。

最后一项特别设计就是前端，即我们使用 DeepSeek Harness 的界面，DSH 并不是一个传统的「固定前端页面 + 后端插件」，界面本身也是第二棵插件树。



DeepSeek Harness (DSH)

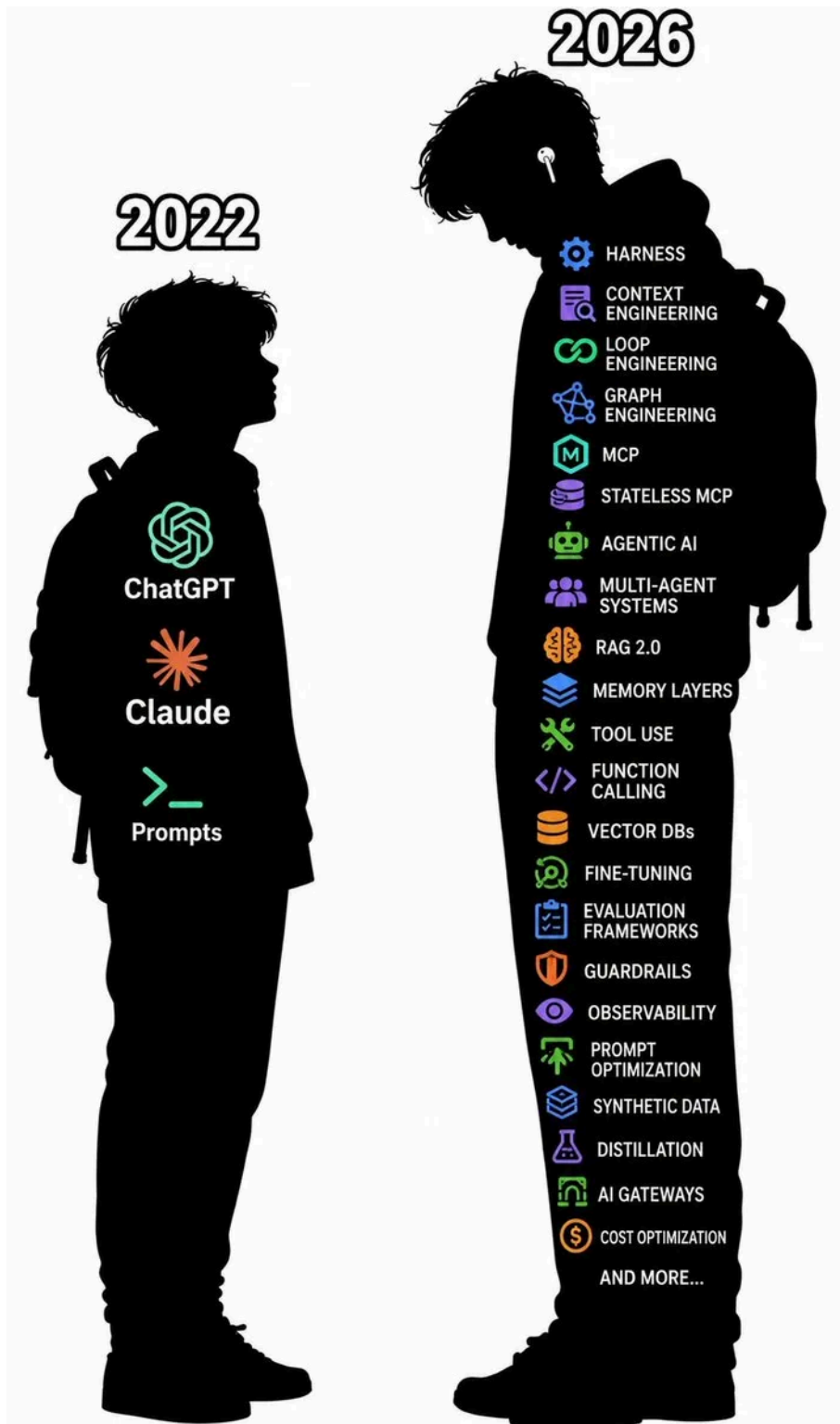
可重组、可回放的 Agent 运行时



应用启动后，界面挂载机制会加载一组 UI 插件。页面先声明侧边栏、对话区、输入区和设置区等标准挂载位，各插件再把自己的组件注册进去。

MCP、Skill、网页搜索、终端、子 Agent、多模型、Plan Mode、Workflow，这些单独看都已经是 Agent 产品的常见能力。

DeepSeek Harness 的不同在于它把模型、工具、会话、插件和界面，都接入统一的底层机制。



而这带来的最大改变，大概是以前我们是使用一个通用 AI Agent，每个人都在用一样的 Codex 去处理办公、编程、写作等任务；但 DeepSeek Harness 给我们的是自己探索的空间。

总的来说，Codex 尝试交付一个拿来即用的 Agent；DeepSeek Harness 更像一套组装 Agent 的运行时：它把更多结构暴露出来，让开发者决定 Agent 应该是谁、使用什么工具，以及如何工作。

既然 Codex 可以切换不同的模型，模型只是 Agent 的一个部件，那 Agent 本身也应该能持续配置、替换和重组。

DeepSeek 过去最受关注的是模型本身。现在到了 DeepSeek Harness，它开始处理模型之外的问题：如何组织工具和上下文、保存执行过程，以及在同一套系统里配置不同的 Agent。

以前我们挑 Agent，挑的是哪家公司做出了最好用的那个产品。

DeepSeek Harness 给出的另一种可能是，**Agent 和模型同样是基础设施**。开发者和用户，可以使用 DeepSeek Harness 创造出无数个 Codex，而真正属于我们的 Agent，也可以由自己组装出来。

本文来源：[APPSO](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

 431  收藏

分享到:  

写评论

请发表您的评论

 表情  图片

发表评论

4 条评论